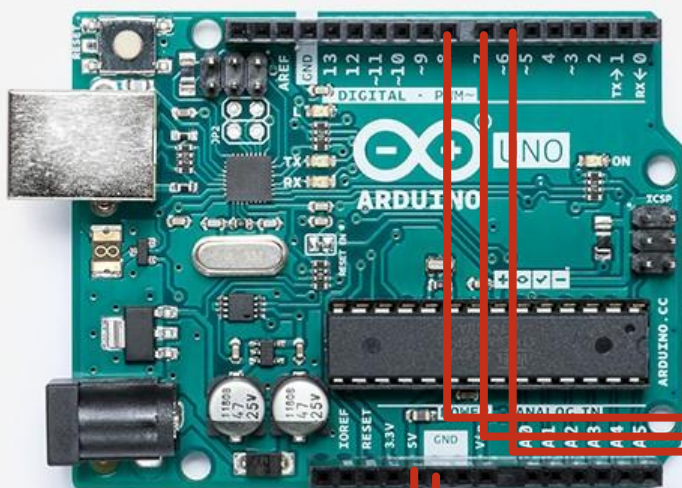


七段顯示器



接digital pin

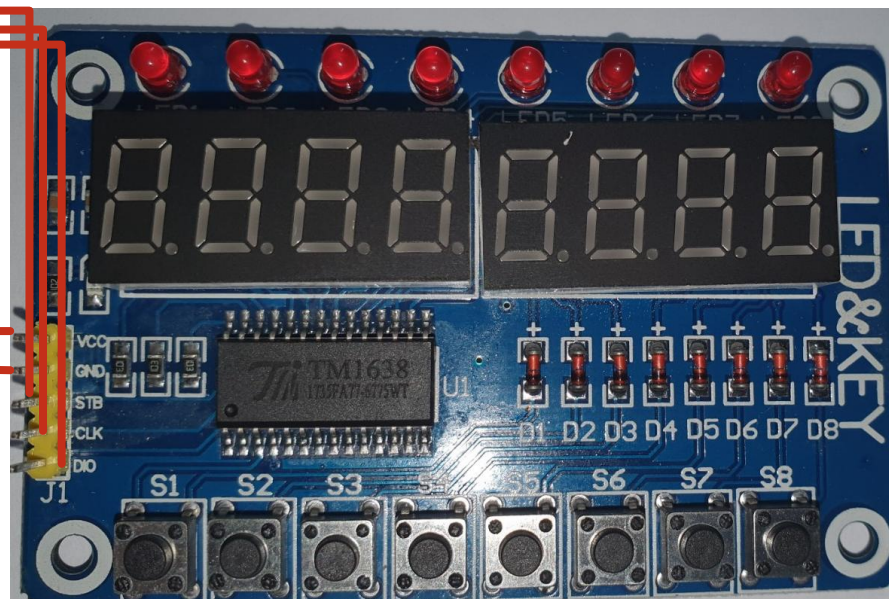
VCC:電源接5V

GND:GND

STB:strobePin

CLK:clockPin

DIO:dataPin



編號 0 1 2 3 4 5 6 7



暫存器

8 bits

8 bits

8 bits

8 bits

8 bits

8 bits

8 bits

8 bits

地址

0xC0

0xC2

0xC4

0xC6

0xC8

0xCA

0xCC

0xCE



command or
data

one pin one bit

Arduino

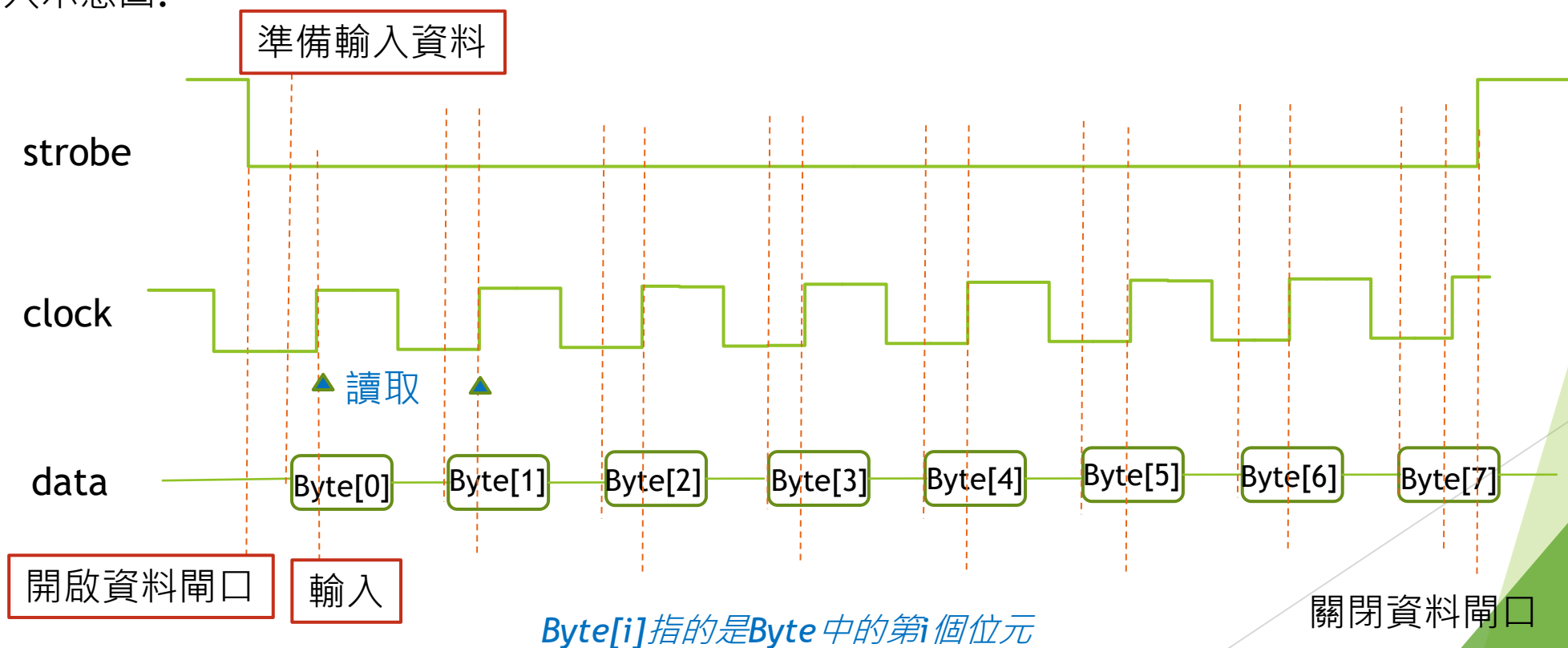
腳位說明

符號	腳位名稱	說明
DIO	數據輸入輸出	1. 在CLK <u>上升</u> (L→H)時輸入數據，輸入的順序從byte[0]到byte[7] 2. 在CLK <u>下降</u> (H→L)時輸出數據，順序同上
CLK	時脈輸入	控制DIO輸入/輸出
STB	片選輸入/輸入閘口	在下降時初始化DIO，隨後等待指令。 在STB為L時接收到的第一個byte為指令。 在STB為H時會忽略CLK
VCC	邏輯電源	接電源正
GND	邏輯地	接電源地

輸入控制

- ▶ **strobePin**: LOW時開啟資料輸入/出閘口，HIGH時關閉
- ▶ **clockPin**: **LOW→HIGH**時讀取**dataPin**
- ▶ **dataPin**: 資料輸入，一次傳一個bit

輸入示意圖：



指令說明

- ▶ 在STB下降沿後由DIO輸入的第一個字節作為一條指令

B7	B6	指令
0	1	數據命令設置
1	0	顯示控制命令設置
1	1	地址命令設置

數據命令設置

- 該指令用來設置數據寫和讀，B1和B2位不允許設置成01或11

B7	B6	B5	B4	B3	B2	B1	B0	功能	說明
0	1	無關項，填0				0	0	數據讀寫模式 設置	寫數據到暫存器
0	1					1	0		讀鍵盤數據
0	1				0			地址增加模式 設置	自動地址增加
0	1				1				固定地址
0	1			0				測試模式設置 (內部使用)	普通模式
0	1			1					測試模式

普通模式自動地址增加：寫數據到暫存器 0x40
普通模式固定地址：寫數據到暫存器 0x44
普通模式自動地址增加：讀鍵盤數據 0x42

數據命令與顯示控制指令

B7	B6	B5	B4	B3	B2	B1	B0	說明
0	1	無關項， 填0				0	0	寫數據到暫存器
0	1					1	0	讀鍵盤數據
1	0			1				打開板子上顯示元件
1	0			0				關閉板子上顯示元件

[illegible]

地址命令

編號 0 1 2 3 4 5 6 7



地址 0xC0 0xC2 0xC4 0xC6 0xC8 0xCA 0xCC 0xCE

B7	B6	B5	B4	B3	B2	B1	B0	暫存器位置	顯示器
1	1	無關項， 填0		0	0	0	0	C0H	0
1	1			0	0	1	0	C2H	1
1	1			0	1	0	0	C4H	2
1	1			0	1	1	0	C6H	3
1	1			1	0	0	0	C8H	4
1	1			1	0	1	0	CAH	5
1	1			1	1	0	0	CCH	6
1	1			1	1	1	0	CEH	7

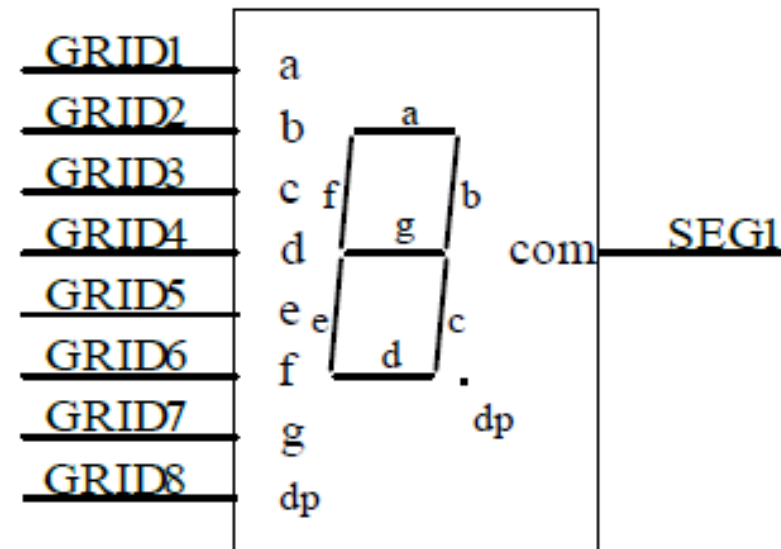
接腳參數

各自的接腳負責控制七段顯示器中每個位置的暗或亮

亮: HIGH(1)

暗: LOW(0)

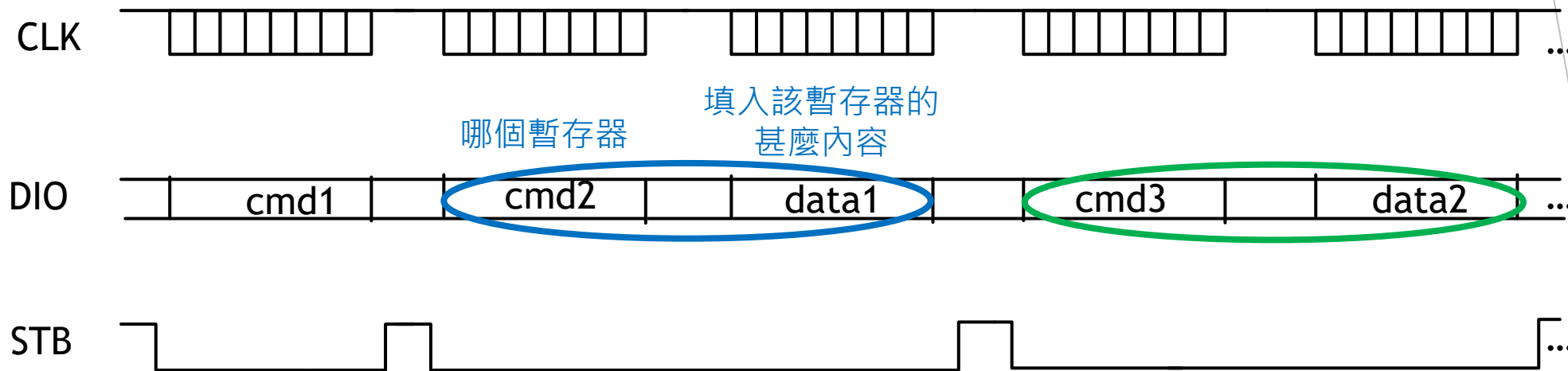
dp:小數點



bit位置		0	1	2	3	4	5	6	7
七段號碼		a	b	c	d	e	f	g	dp
數字0~3	0	1	1	1	1	1	1	0	0
	1	0	1	1	0	0	0	0	0
	2	1	1	0	1	1	0	1	0
	3	1	1	1	1	0	0	1	0

0x3f

固定地址傳輸模式



- **cmd1:**設定數據命令
- **cmd2:**設定暫存器位置
- **data1:**傳輸data1給cmd2指定的暫存器
- **cmd3:**設定暫存器位置
- **data2:**傳輸data2給cmd3指定的暫存器

Command	編碼
寫數據到暫存器 自動地址增加	(0x40)
寫數據到暫存器 固定地址	(0x44)
讀鍵盤數據	(0x42)
打開板子上顯示元件	(0x88)
關閉板子上顯示元件	(0x80)

暫存器位置	顯示器
0xC0	0
0xC2	1
0xC4	2
0xC6	3
0xC8	4
0xCA	5
0xCC	6
0xCE	7

模組設計

sendByte()：傳送一個byte的內容

```
void sendByte(byte data) // 參數data 1個byte的使用者資料
{
    byte right_most_bit;
    for (int i = 0; i < 8; i++) { // 產生八個clock方波(週期)
        digitalWrite(clockPin, LOW);
        right_most_bit = data & 1; // 利用datapin一次輸出最右邊一個data bit
        if(right_most_bit > 0)
            digitalWrite(dataPin, HIGH );
        else
            digitalWrite(dataPin, LOW );
        data = data >> 1; // right shift one bit
        digitalWrite(clockPin, HIGH);
    }
}
```

$$\begin{array}{r} 01001101 \\ \& 00001010 \\ \hline 00001000 \end{array}$$
$$\begin{array}{r} 0100110X \\ \& 00000001 \\ \hline 0000000X \end{array}$$

data = data >> 1;
全部往右移一個bit
01001101
變成
00100110

模組設計 (進階版)

sendByte()：傳送一個byte的內容

```
void sendByte(byte data) // 參數data 1個byte的使用者資料
{
    for (int i = 0; i < 8; i++) { // 產生八個clock方波(週期)
        digitalWrite(clockPin, LOW);

        digitalWrite(dataPin, (data & 1 ? HIGH : LOW)); //利用datapin一次輸出最右邊一個data bit
        data >>= 1;

        digitalWrite(clockPin, HIGH);
    }
}
```

說明

(data & 1 ? HIGH : LOW)

等於

if(data & 1 == 1) // bitwise AND

temp = HIGH;

else

temp = LOW;

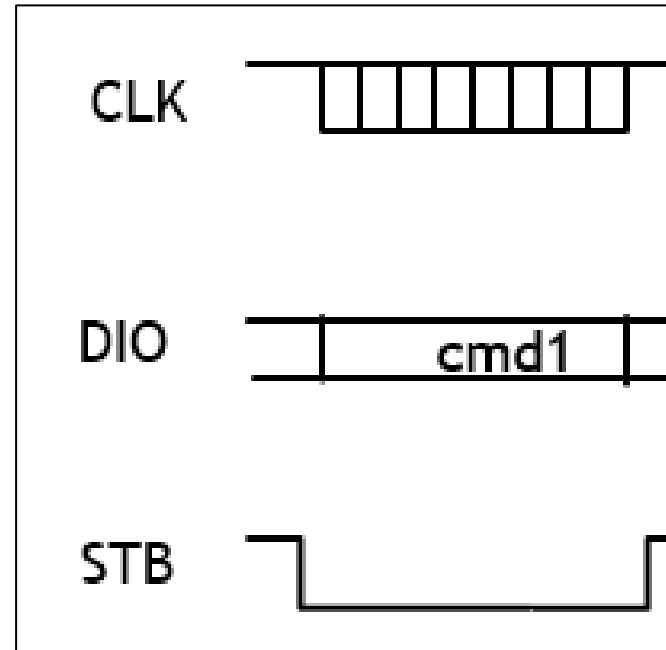
說明

data >> 1;等於data = data >> 1;

模組設計

sendCommand()：利用**sendByte()** 傳送一個byte的指令

```
void sendCommand(byte cmd) //設定strobe以輸入1個byte的資料
{
    digitalWrite(strobePin, LOW);
    sendByte(cmd); //右圖中cmd1
    digitalWrite(strobePin, HIGH);
}
```



初始設定

```
void setup() {
```

```
    pinMode(dataPin, OUTPUT);
```

```
    pinMode(clockPin, OUTPUT);
```

```
    pinMode(strobePin, OUTPUT);
```

```
    digitalWrite(strobePin, HIGH);
```

```
    digitalWrite(clockPin, HIGH);
```

設定各個腳位為output

將strobe跟clock設為HIGH以便後續sendCommand

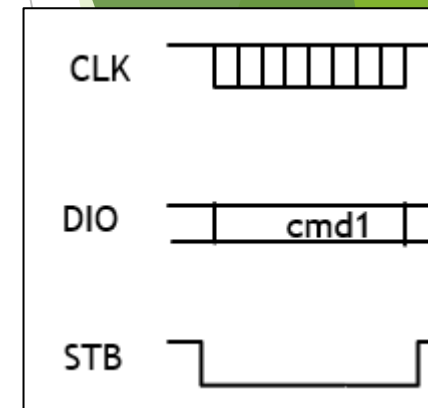
```
    sendCommand(0x40);
```

設定為”數據寫入模式”

```
    sendCommand(0x88);
```

開啟板子上的顯示元件

```
}
```



說明

寫數據到暫存器 (自動地址增加) (0x40)

寫數據到暫存器 (固定地址) (0x44)

讀鍵盤數據 (0x42)

打開板子上顯示元件 (0x88)

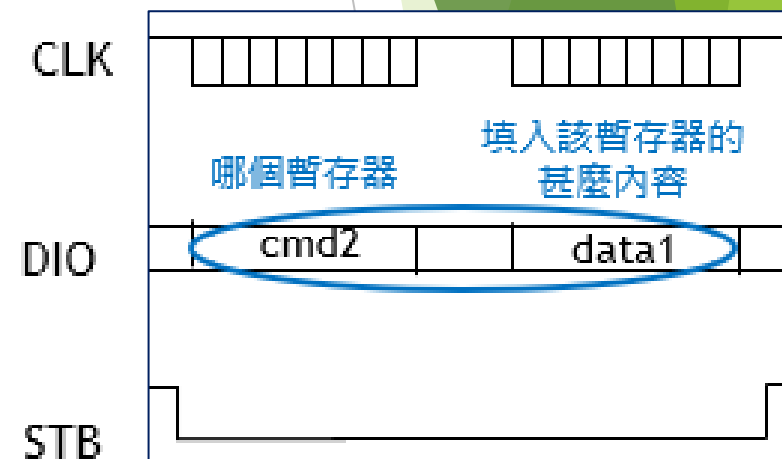
關閉板子上顯示元件 (0x80)

模組設計

setSegNum()：寫入圖形到某個七段顯示器上

```
void setSegNum(byte seg_address, byte pattern) //seg_address: 七段顯示器暫存器的位址
{
    //pattern: 要對這個暫存器寫入甚麼內容

    digitalWrite(strobePin, LOW); //開始進行輸入
    sendByte(seg_address); //cmd2: 指定寫入的七段顯示器位置
    sendByte(pattern); //data1: 寫入要顯示的圖案內容
    digitalWrite(strobePin, HIGH); //結束輸入
}
```



- ▶ seg_address: 0b1100XXX0(0xcX) 其中XXX指定開始顯示的位置，顯示器由左到右分別是0~7(二進位)
- ▶ pattern: 8個bits[7:0]依序是 d p g f e d c b a
- ▶ 如果七段顯示器的內容不進行清空(寫入0x00)，就會顯示之前的結果

範例程式:

在最左邊的七段顯示器中顯示0

暫存器位置	顯示器
0xC0	0
0xC2	1
0xC4	2
0xC6	3
0xC8	4
0xCA	5
0xCC	6
0xCE	7

```
digitalWrite(strobePin, LOW);
sendByte(0xC0);    // 最左邊的七段顯示器
sendByte(0x3F);    // 寫入0的圖形內容
digitalWrite(strobePin, HIGH);
```



```
setSegNum(0xc0, 0x3f)
```

